

Ревью кода. Модульная структура мобильного приложения

Принцип разбиения на модули

В мобильном приложении СБИС выделяем 4 обязательных уровня:

- само приложение (уровень Application);
- прикладные модули (коммуникатор, меню, логин, уведомления и другие...) (Operational);
- декларативный уровень (Declaration);
- уровень общих библиотек (Library).

1 уровень Application (Приложение)

Это уровень самого приложения. Связующий модуль. Необходим всегда, для сборки приложения из нескольких прикладных модулей. Не содержит реализацию какой-либо БЛ.

Реализует инициализацию приложения и модулей согласно их сценариям инициализации. Предоставляет интерфейс для модулей для получения реализации протоколов.

Репозитории этого уровня:

- <https://git.sbis.ru/mobileworkspace/ios-app> Приложение iOS
- <https://git.sbis.ru/mobileworkspace/android-app> Приложение Android

2 уровень Operational (Прикладные модули)

На данном уровне сосредоточена бизнес-логика приложения. Здесь содержатся такие модули как «Коммуникатор», «СБИСДиск», «Авторизация», «Меню», «Уведомления», «Новости» и т.д. . Обращаю внимание что модули этого уровня не используют прямую зависимость друг от друга.

Репозитории этого уровня. iOS:

- Login: <https://git.sbis.ru/mobileworkspace/ios-login>
- Коммуникатор: <https://git.sbis.ru/mobileworkspace/ios-communicator>
- СБИСДиск: <https://git.sbis.ru/mobileworkspace/ios-disc>
- Меню <https://git.sbis.ru/mobileworkspace/ios-menu>

Android:

- Login: <https://git.sbis.ru/mobileworkspace/android-login>
- Коммуникатор: <https://git.sbis.ru/mobileworkspace/android-communicator>
- СБИСДиск: <https://git.sbis.ru/mobileworkspace/android-disc>
- Уведомления: <https://git.sbis.ru/mobileworkspace/android-notification>
- Меню: <https://git.sbis.ru/mobileworkspace/android-menu>

3 декларативный уровень (Declaration)

Любое общение модулей между собой осуществляется только посредством интерфейсов. Объявление этих интерфейсов и есть цель декларативного модуля. Если есть модель, которая используется более чем в одном модуле, она тоже должна быть объявлена здесь. По факту этот модуль является своего рода протоколом общения между модулями 1 и 2 уровня.

Любое взаимодействие модулей из 2 уровня между собой осуществляется через этот уровень. Прямое использование реализации запрещено.

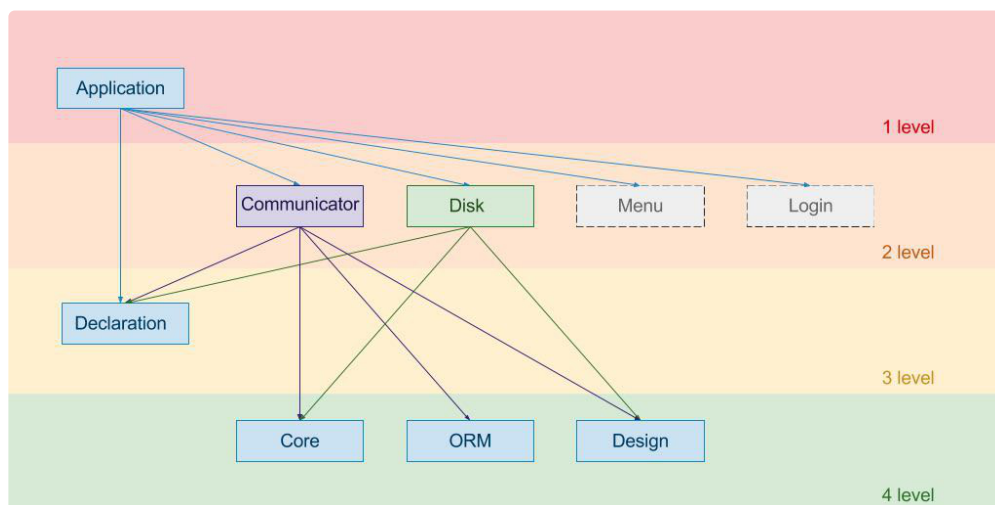
Репозитории этого уровня:

- <https://git.sbis.ru/mobileworkspace/android-serviceAPI> (Андроид)
- <https://git.sbis.ru/mobileworkspace/ios-serviceAPI> (iOS)

4 уровень Library (Общие библиотеки)

На данном уровне находятся модули, которые предоставляют общий служебный функционал для прикладных модулей уровня 2. Сюда относятся такие модули как Design (общие элементы дизайна для приложения, стили, View), Core (модуль работы с методами БЛ), Utils (Общие функции не связанные с дизайном интерфейса или платформой).

Взаимозависимость модулей



Модуль Application является модулем-сборщиком приложения. В его зоне ответственности инициализировать все модули 2-ого уровня. Только данный модуль знает конечную реализацию модуля, только модуль Application является зависимым от любого модуля 2-ого уровня.

Модули 2-ого уровня не имеют физической зависимости друг от друга. Взаимодействие модулей, осуществляется посредством интерфейсов(протоколов), объявленных на уровне 3.

Модуль Declaration не имеет зависимостей и является базовым.

От него зависят модули уровня 1 и 2.

Модули 4-ого уровня являются базовыми и не имеют зависимостей.

Таким образом, для сборки минимального приложения требуется подключить модули 4 и 3 уровня. Подключение всех модулей из уровня 2 не обязательно. Что позволяет, например, сделать отдельные отладочные приложения для модулей бизнес-логики.

Версионность модулей

У каждого модуля есть своя версия сборки. И если модуль зависит от другого модуля, он так же зависит и от версии этого модуля. Может так случиться, что два модуля зависят от одного модуля разных версий. Возникает конфликт. Для решения таких конфликтов зависимые модули обычно указывают диапазон версий с которыми они способны работать.

Например:

> 0.3.2 требуется версия не ниже 0.3.2

<= 0.5 требуется версия не более 0.5

В данном случае общей рабочей версией зависимого модуля может быть любая сборка от версии 0.3.2 по версию 0.5.

Для управления версиями модулей на iOS используем CocoaPods репозиторий, находящийся по адресу:

<https://git.sbis.ru/mobileworkspace/specs>

[Документация по использованию CocoaPods репозитория и оформления модулей.](#)

Для Android развернут maven-репозиторий по адресу maven.sbis.ru.

[Документация по использованию Maven-репозитория.](#)

Принцип построения модуля уровня 2 (БЛ)

Трехзвенная архитектура модуля

Мобильное приложение строится по трехзвенной архитектуре:

1. Серверная бизнес-логика (БЛ) модуля – описывается набором веб-сервисов, которые отдают API на основе sbis-grs.
2. Клиентская бизнес-логика модуля – реализуется как набор общих библиотек, занимается вызовом серверной бизнес-логики, обработкой и хранением данных, полученных от серверной БЛ. Обеспечивает всю работу приложения в режиме, когда серверная БЛ недоступна (офлайн).

3. Графический интерфейс пользователя (GUI) – GUI не делает прямые вызовы к серверной БЛ. Чтобы получить или изменить данные, этот слой всегда обращается к клиентской БЛ.

По сути, внутри модуль построен по архитектуре клиент-сервер. UI часть модуля сама не осуществляет какой-либо работы с данными а делает запрос в БЛ приложения для построения ViewModel.

